Secret Sharing

(core constructions and techniques)

Trivial Secret Sharing

Goal

Secret: 00101101

How to share a secret among two persons such that it is revealed if BOTH reveal their part?

Very first idea

Secret: 0010.1101

Alex: 0010----

Bob: ----1101

Wise idea???

Bad idea!

Which probability to guess secret right?

- before seeing anything : 1/256
- after seeing one share : 1/16

Secret WEAKENED when one share revealed

Alex: 0010----

Bob: ----1101

Unavoidable?
Can we do better?

A better solution

Secret: 0010.1101

Generate random sequence, ex. 1011.0100

XOR Secret & random 1001.1001

Give alex the random sequence

1011.0100

Give Bob the XOR value

1001.1001

Secret = Ashare XOR Bshare

1011.0100 ⊕ 1001.1001 = 0010.1101

Is this secure?

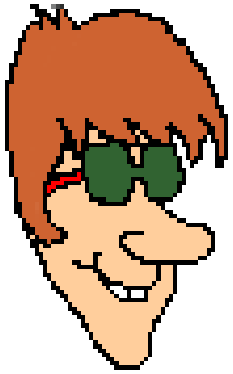
Security / A

Alex: 1011.0100

Eve gets to know Alex' SHARE:
NO INFORMATION ON SECRET REVEALED

Alex share is a random sequence!
No secret included in his share!!

Security / B



Eve gets to know Bob' SHARE:

Bob share is an XOR between random sequence and the secret…

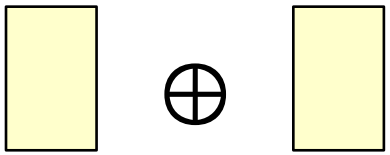
Bob:1001.1001

Does this leak information?

Perfect secrecy

➔Of course no information leaked

⇨ Remember one time pad…



Secret bit

Probability:
0=p
1=(1-p)

Random bit

Probability:
0=1/2
1=1/2

Secret bit	Random bit	XOR result bit	
0	0	0	p/2
0	1	1	p/2
1	0	1	(1-p)/2
1	1	0	(1-p)/2

Probability to guess secret after seeing ONE share is the same as a guess without seeing any share

XOR not strictly needed: Can use (modular) sums

SECRET 0010.1101→ 45

RAND mod 256: 180

S – RAND mod 256: 121

180

121

Secret = Ashare + Bshare

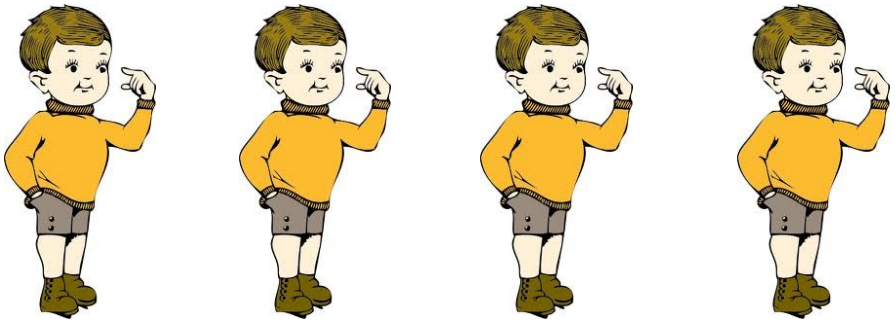
180+121 mod 256 = 45

Module not necessary, but practical (e.g. 32 bit numbers)

Trivially extends to n parties

S → 45
R1 → 135
R2 → 6
R3 → 67
S-R1-R2-R3 → 93

135 6 67 93



Reconstruction:
$(135+6+67+93) \bmod 256 = 45$

Perfect secrecy up to n-1 shares revealed

adversary knowing n-1 shares has still (initial!) 1/256 prob to guess secret

Shamir Secret Sharing

Further goal

- **(n,n) secret sharing scheme**
  - Secret revealed only if **ALL n** parties provide a share
  - The previous trivial scheme

- **(t,n) secret sharing scheme**
  - Secret revealed when **ANY t OUT of n** parties provide a share
    - $1 \leq t \leq n$
    - If t=1, every partner has secret (trivial case)

Why (t,n) schemes?

Start of Shamir's paper *- Eleven scientists are working on a secret project. They wish to lock up the documents in a cabinet so that the cabinet can be opened if and only if six or more of the scientists are present.*

What is the smallest number of locks needed? → **462**

What is the smallest number of keys to the locks each scientist must carry? → **252**

➔ **Compelling real world scenarios**

⇨ two-out-of-tree nuclear weapon control in Russia in the early 1990s

→ President, Defense Minister, and Defense Ministry [Beimel 2010].

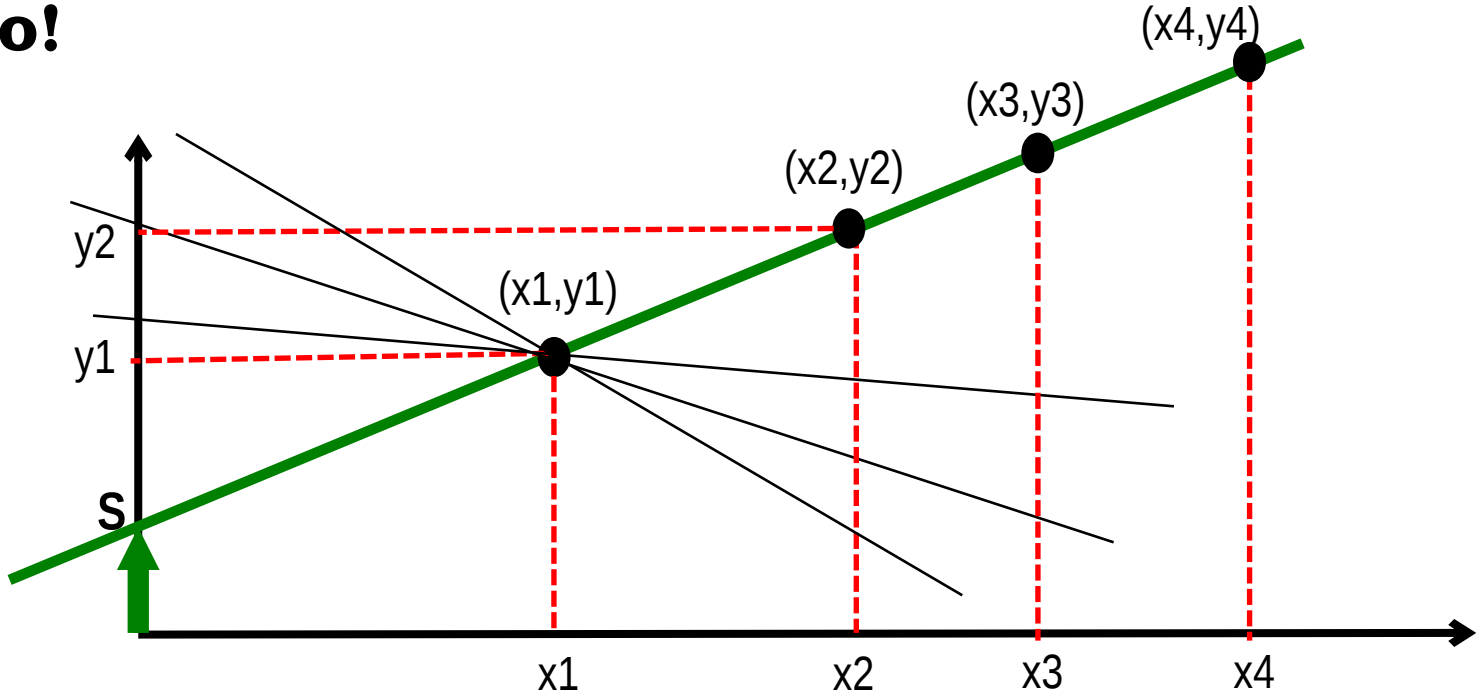
➔ **Basic building block in several cryptographic constructions**

➔ **Shamir 1979, Blakley 1979**

⇨ Independently, different solutions

A (2,n) scheme: idea

- **How many points uniquely define a line?**
- **Two!**



- **Secret S = y-value at x=0**
- **Share for user i: point (x_i,y_i)**
  - One share: S remains unknown
  - Two+ shares: Permit to UNIQUELY determine S!

Procedure: dealing

- **Dealer: construct line s. t.**
  - coefficient a: randomly chosen
  - Secret S: known term

$$y = S + a \cdot x$$

- **Example: a=15, S=39**

$$y = 39 + 15 \cdot x$$

- **Distributes shares to n participants**
  - x_i randomly chosen or a priori known
    - Participant 1: $x_1 = 1$ → share = (1, 54)
    - Participant 2: $x_2 = 2$ → share = (2, 69)
    - Participant 3: $x_3 = 3$ → share = (3, 84)
    - …

Procedure: reconstructing

- **Receive two shares** $P_i = (x_i, y_i)$ $P_j = (x_j, y_j)$

- **Interpolate points (equation of line)** $\frac{y - y_j}{y_i - y_j} = \frac{x - x_j}{x_i - x_j} \Rightarrow y = y_j + \frac{x - x_j}{x_i - x_j}(y_i - y_j)$

- **Set x=0 for deriving y=S** $S = y_j + \frac{0 - x_j}{x_i - x_j}(y_i - y_j) = y_j \frac{-x_i}{x_j - x_i} + y_i \frac{-x_j}{x_i - x_j}$

- **Example: Pi=(3,84), Pj=(1, 54)**

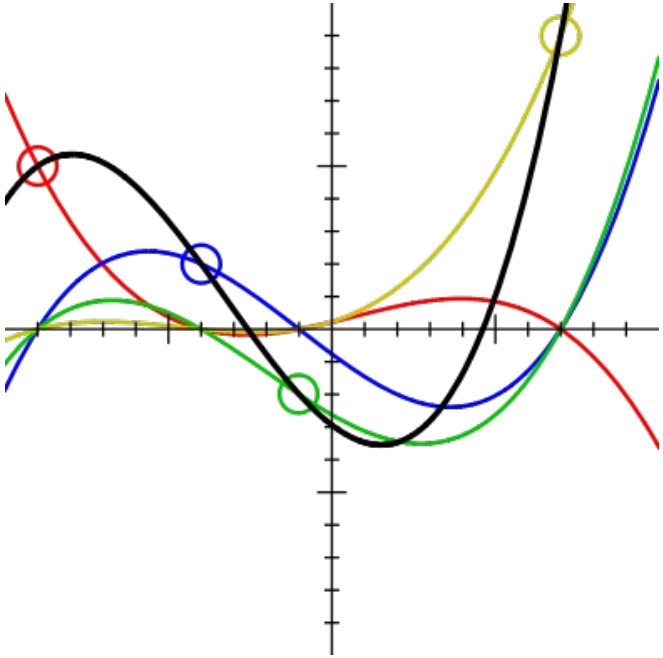
$$\frac{y - 54}{84 - 54} = \frac{x - 1}{3 - 1} \Rightarrow y = 54 + 15(x - 1) \Rightarrow S = 54 + 15(0 - 1) = 54 - 15 = 39$$

$$S = y_1 \frac{-3}{1 - 3} + y_3 \frac{-1}{3 - 1} = 54 \cdot \frac{3}{2} - 84 \cdot \frac{1}{2} = 39$$

Extension to (t,n)

- **Polynomial of degree t-1 is uniquely defined by t points**
  - Line → 2 points
  - quadratic → 3 points
  - Cubic → 4 points
  - …
- **Need formula to interpolate t shares**
- **Most convenient: Lagrange interpolation**

Reminder: Lagrange basis polynomials and Lagrange Interpolation



➔ **Any polynomial of degree t-1 with t known points (x_1,y_1), …, (x_t,y_t) can be decomposed as**

$$y = \sum_{i=1}^{t} y_i \Lambda_i(x)$$

➔ **Where $\Lambda_i(x)$ is the basis polynomial**

$$\Lambda_i(x) = \prod_{m=1, m\neq i}^{t} \frac{x - x_m}{x_i - x_m} = \frac{(x - x_1)}{(x_i - x_1)} \cdots \frac{(x - x_{i-1})}{(x_i - x_{i-1})} \frac{(x - x_{i+1})}{(x_i - x_{i+1})} \cdots \frac{(x - x_t)}{(x_i - x_t)}$$

➔ **Clearly a basis:** $\Lambda_i(x_i) = 1; \quad \Lambda_i(x_m) = 0 \quad for\ m \neq i;$

(t,n) scheme

- **Dealer:**
  - Generate random polynomial p(x) with degree (t-1)
  - Set secret s as known term in the polynomial

$$p(x) = s + a_1 x + a_2 x^2 + \cdots + a_{t-2} x^{t-2} + a_{t-1} x^{t-1}$$

  - Distribute one share to each of the n parties

$$(x_i, y_i) \quad y_i = p(x_i)$$

- **Reconstruction**
  - Collect t shares out of the n available
  - Compute secret using Lagrange interpolation @ x=0

$$s = \sum_{shares\ x_i} y_i \Lambda_{x_i} \quad with \quad \Lambda_{x_i} = \Lambda_{x_i}(0) = \prod_{shares\ x_k \neq x_i} \frac{-x_k}{x_i - x_k}$$

Example: (3,4) scheme

Secret: value between 1 and 100

$$\text{dealer}: s = 32$$

$$\text{dealer}: p(x) = 32 + 52x + 3x^2$$

$$\text{shares}: (1,87), (2,148), (3,215), (4,288)$$

Collected shares: 1,2,3 (4 missing)

$$\Lambda_1 = \frac{-x_2}{x_1 - x_2} \cdot \frac{-x_3}{x_1 - x_3} = \frac{-2}{1-2} \cdot \frac{-3}{1-3} = 3$$

$$\Lambda_2 = \frac{-x_1}{x_2 - x_1} \cdot \frac{-x_3}{x_2 - x_3} = \frac{-1}{2-1} \cdot \frac{-3}{2-3} = -3$$

$$\Lambda_3 = \frac{-x_1}{x_3 - x_1} \cdot \frac{-x_2}{x_3 - x_2} = \frac{-1}{3-1} \cdot \frac{-2}{3-2} = 1$$

$$\text{secret}: s = y_1\Lambda_1 + y_2\Lambda_2 + y_3\Lambda_3 = 87 \cdot 3 + 148 \cdot (-3) + 215 \cdot 1 = 32$$

What about secrecy?

- **Unconditionally secure scheme:**
  - knowledge of any t-1 or fewer shares leaves secret s undetermined
    - in the sense that all its possible values are equally likely

- **Is this true???**
- **NO! Scheme presented so far is flawed!!**

Example: shares 1 and 3 collected; can we say something about s??

compute as before : $\Lambda_1 = 3; \Lambda_2 = -3; \Lambda_3 = 1$

share 2 unknown : set to D

secret : $s = y_1\Lambda_1 + D\Lambda_2 + y_3\Lambda_3 = 87 \cdot 3 + D \cdot (-3) + 215 \cdot 1 = 476 - 3D$

D=integer → s=98 (D=126), 95, 92, 89, 86, 83, … 2/3 of values s EXCLUDED!

Knowledge of shares 1 and 3 permits to triplicate guess probability!

The real Shamir scheme!!

- **Use modular arithmetic**
  - instead of real arithmetic
  - Secret & polynomial in prime field F_p
    - Interpolation formula still holds mod p

- **Result: unconditionally secure scheme!**

- **Which condition on prime p?**
  - Just greater than the domain for the secret!
    - E.g. if secret in (0,100), p=101 perfectly OK
  - Does NOT strictly need to be large!
    - Security is NOT computational (related to some hard problem) but is information theoretic!

Example – mod p=101

Secret: value between 1 and 100

$$\text{dealer}: s = 32$$

$$\text{dealer}: p(x) = 32 + 52x + 3x^2 \mod p = 101$$

$$\text{shares}: (1,87), (2,47), (3,13), (4,86)$$

Collected shares: 1,2,3 (4 missing)

$$\Lambda_1 = 3; \Lambda_2 = -3; \Lambda_3 = 1;$$

$$s = y_1\Lambda_1 + y_2\Lambda_2 + y_3\Lambda_3 = 87 \cdot 3 + 47 \cdot (-3) + 13 \mod 101 = 32$$

$$s = 87 \cdot 3 + D \cdot (-3) + 13 \mod 101 = 72 - 3D \mod 101$$

S uniformly distributed in F_p!

Ideality

- **Can share be smaller than secret?**
  - No: share at least as large as secret
  - Intuition: Given t-1 shares, no information can be determined about secret. Thus, final share must contain as much information as the secret itself.

- **Shamir scheme: ideal!**
  - |share|=|secret|
  - Few proposed schemes NON ideal
    - share larger than secret (e.g. t-times, n-times)
    - E.g Blakley scheme shares are t-times secret

Secret Sharing for Secure Multiparty Computation (basics)

Homomorphic property

Secret S_F

Polynomial f(x)

(1,f(1)) (2,f(2)) (3,f(3)) (4,f(4))




(1,g(1)) (2,g(2)) (3,g(3)) (4,g(4))

Secret S_G

Polynomial g(x)

Homomorphic property:

SUM of the SHARES = Share of the SUM S_F+S_G

f(i), g(i) → f(i)+g(i) → (f+g)(i)

Revealing t composite shares reveals SUM of secrets but does NOT reveal individual secrets

What is SMC?

- **Secure Multiparty Computation**
  - Compute the result of a function without revealing input data
- **Several use-case scenarios**
  - Business/financial, security, medical, traffic monitoring, etc
  - Largely underrated in the real-world, IMHO
    - (also for historical reasons: considered “cumbersome”)
- **Main facts**
  - Born as 2-party computation (Yao’s millionaire problem, 1982)
  - VERY general theorems available
    - ANY ARBITRARY 2-party (Yao, 1986) and multiparty (Goldreich, Micali, Widgerson, 1987) computation can be made secure
    - But cost of construction may be non practical

More specifically

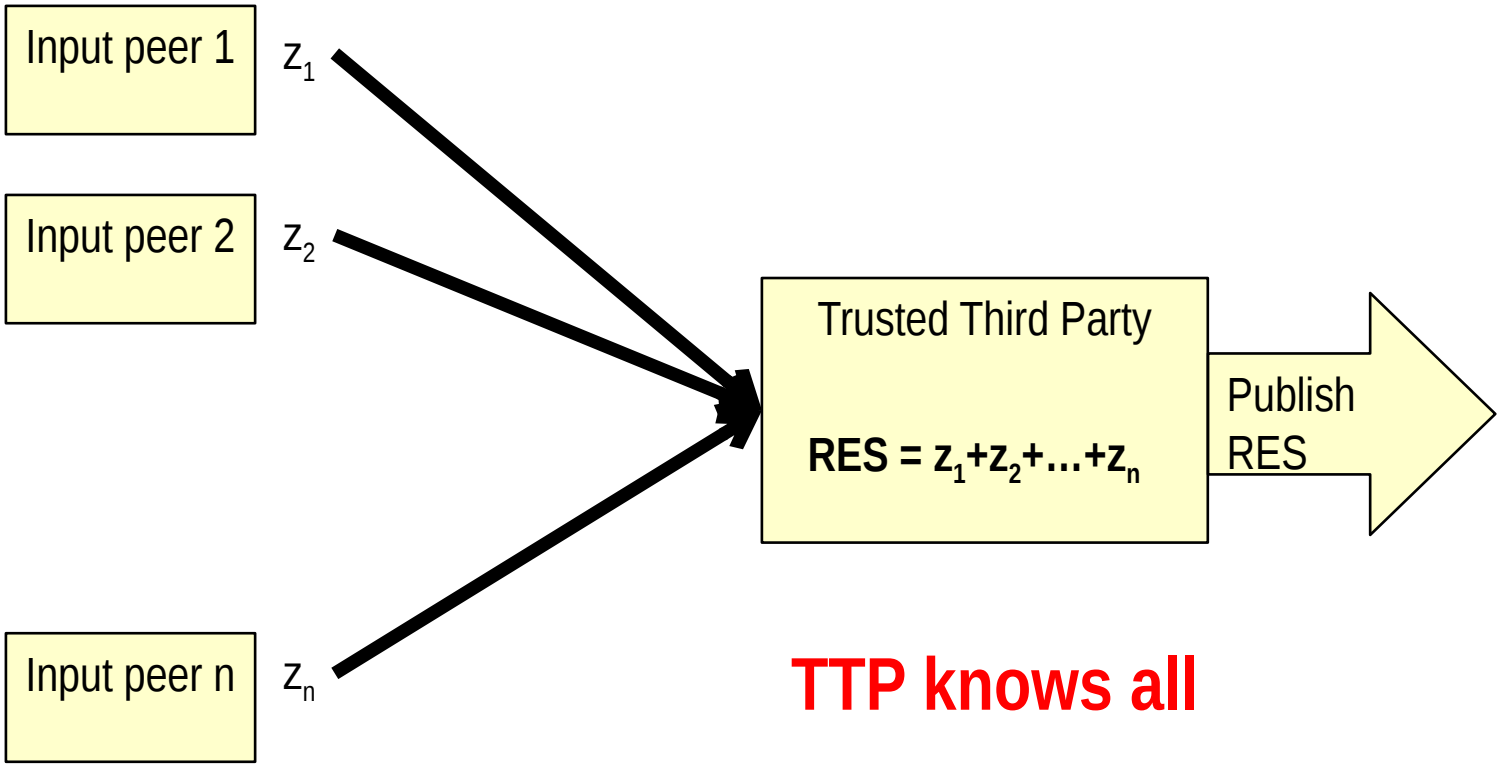
- **N parties P_1, P_2, …, P_n**
  - Each with value z_i

- **Compute function $f(z_1, z_2, \ldots, z_n)$ s.t.**
  - Result is public
  - But NO information is given on input values z_i

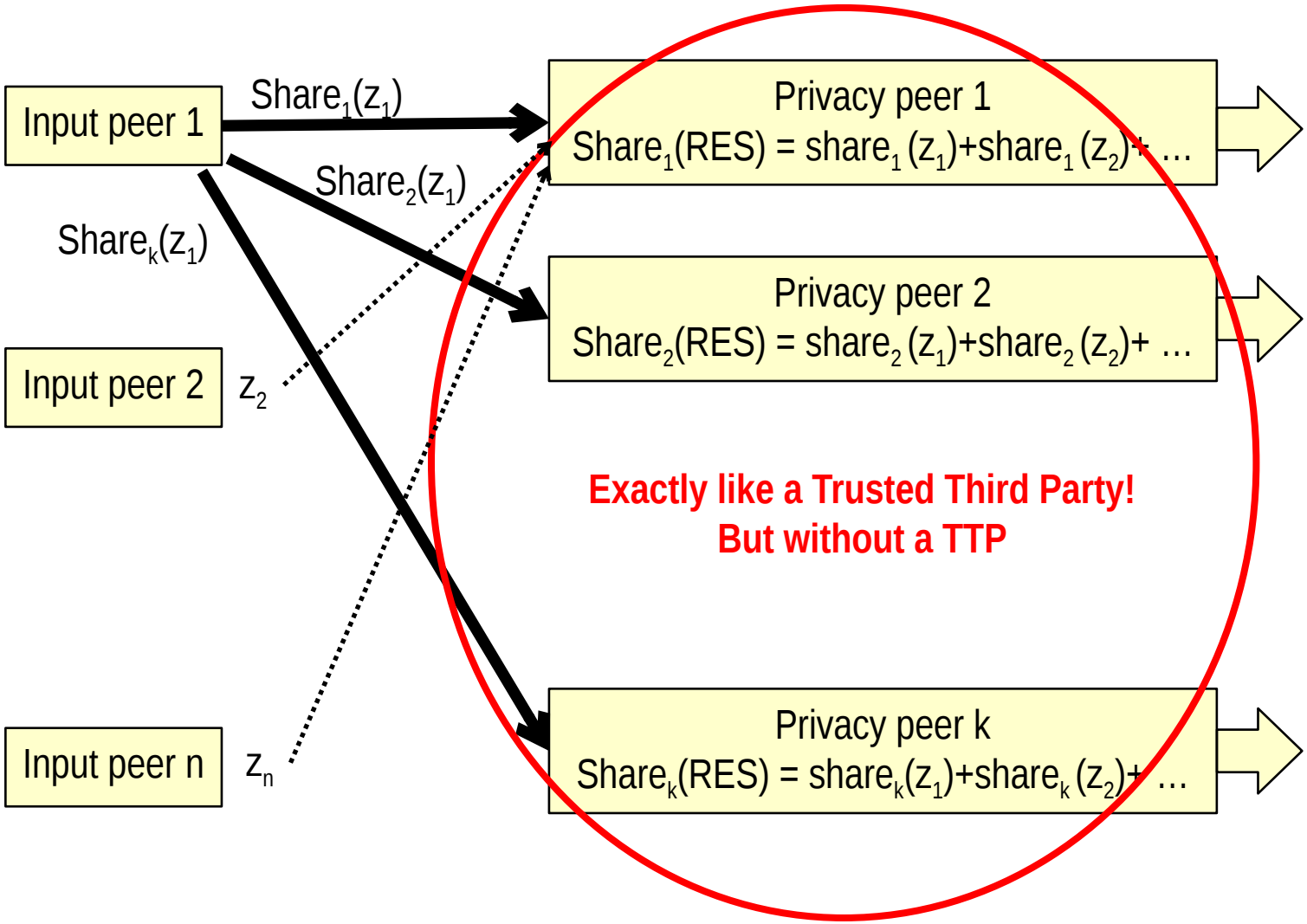
- **Our interest in the special case of addition**
  - Statistics aggregation
  - Actually, works also for weighted addition
    - $f(z_1, z_2, \ldots, z_n) = \alpha_1 z_1 + \alpha_2 z_2 + \ldots + \alpha_n z_n$
  - Very frequent case in practice! More than what you imagine

- **If f=addition, then very efficient SMC from secret sharing schemes!**

Without SMC: Third party



With SMC: "simulated TTP"!



Deployment issues

- **Input peers n:**
  - At least 3; If 2, one party may know other party data by subtracting from final result
  - Can be MANY
    - Thousands of end users
- **Privacy peers k**
  - At least 2
    - But the more, the better for security
    - More robust to collusion
  - Usually small number
  - Threshold # peers: $2 \leq t \leq k$: if (t,k) scheme
- **Input peers may be privacy peers**
  - No change in construction

Construction

- **Input peer i**
  - Input data z_i
  - generate polynomial $p_i(x)$ of degree t-1 with z_i as known term
  - Privately send shares $p_i(1),\ldots,p_i(k)$ to privacy peer 1,…,k
    - No coordination across input peers!!

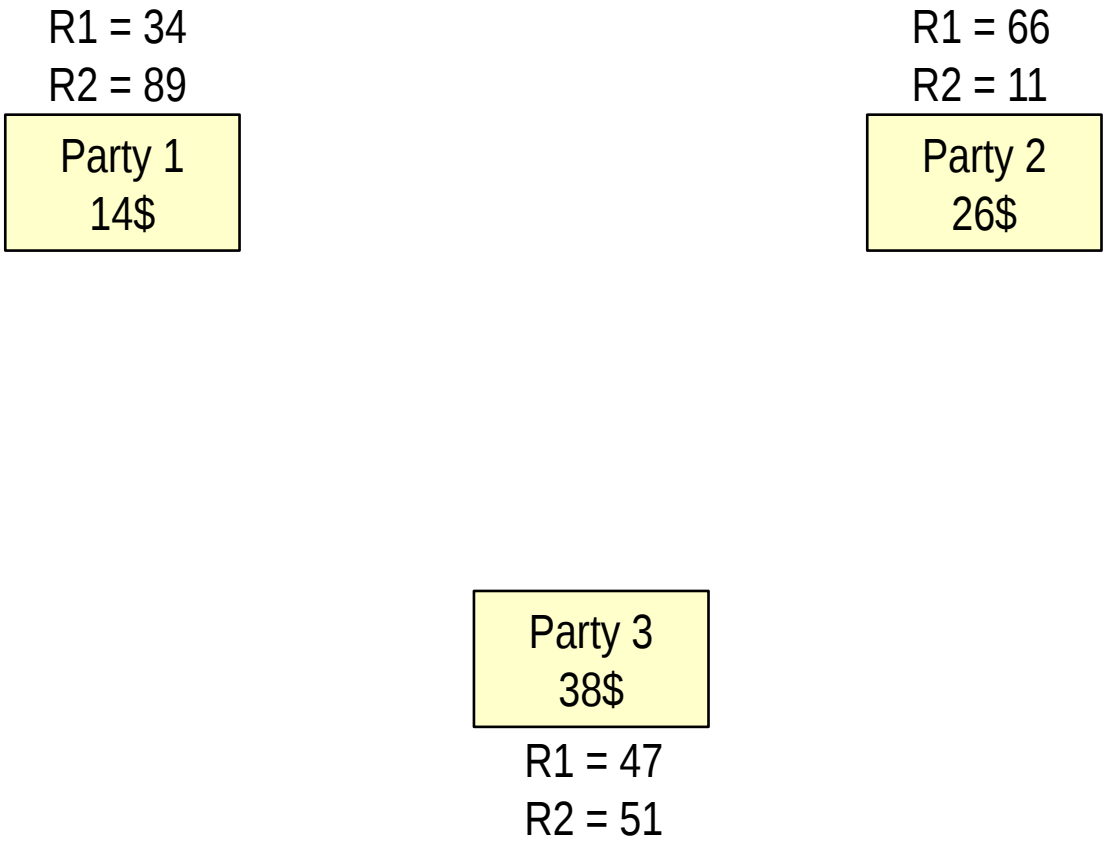
- **Privacy peer m**
  - Collect input shares $p_1(m), p_2(m), \ldots, p_n(m)$
  - Compute $RES(m) = p_1(m) + p_2(m) + \ldots + p_n(m)$
  - Publish aggregated share RES(m)

- **Public:**
  - Reconstruct RES from sufficient number of RES(m)

Using all privacy peer? Much faster!

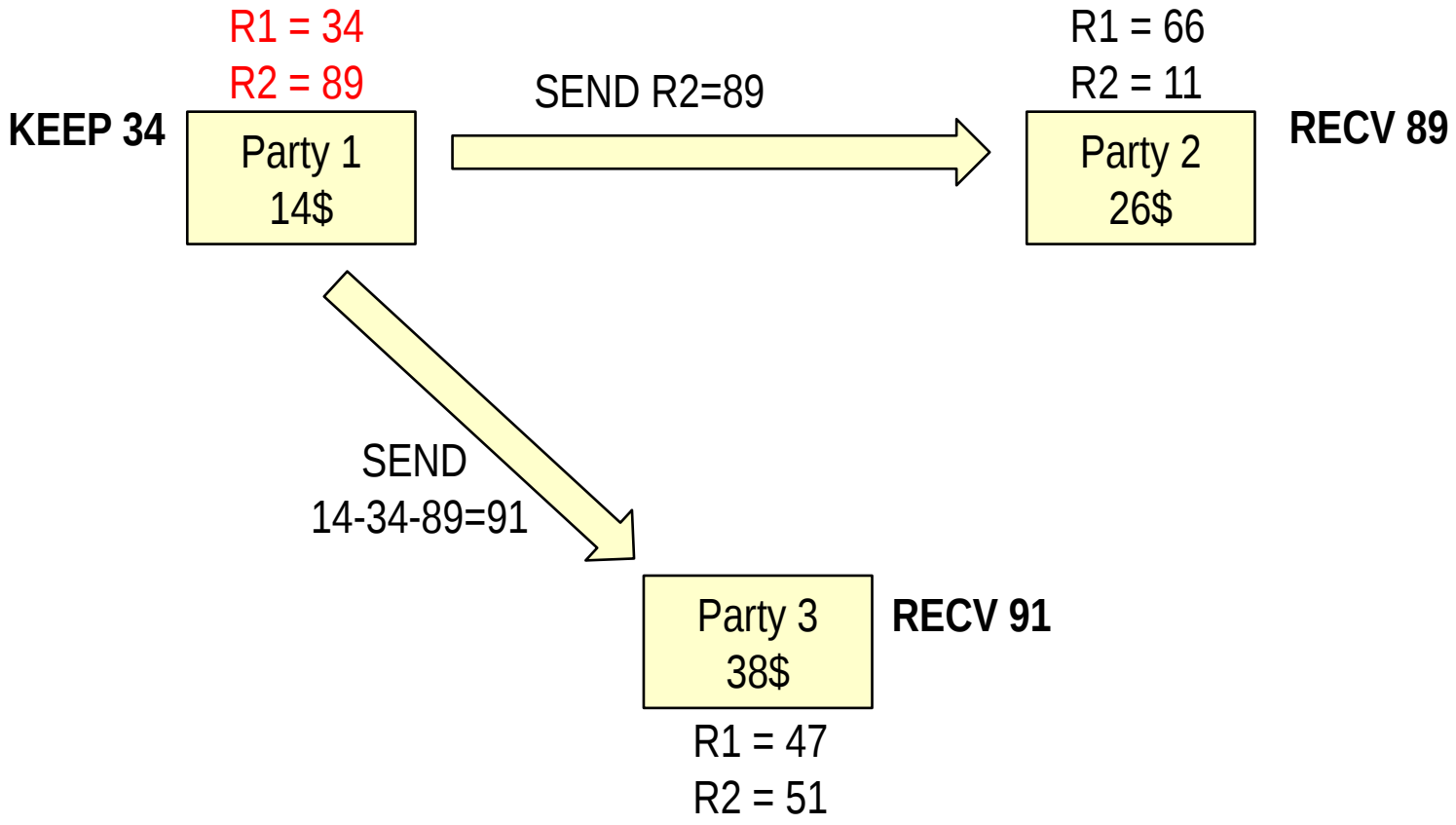
- **Roll back to (k,k) trivial secret sharing!**
  - Measurement z
  - Share 1: r1
  - Share 2: r2
  - …
  - Share k: s-r1-r2-..
- **Ordinary modular arithmetics, on small non-prime fields**
  - E.g. usual 32 bit CPU word
  - No need to “change” any software
  - No performance impairment at all!!

Example
Imput peers = privacy peers



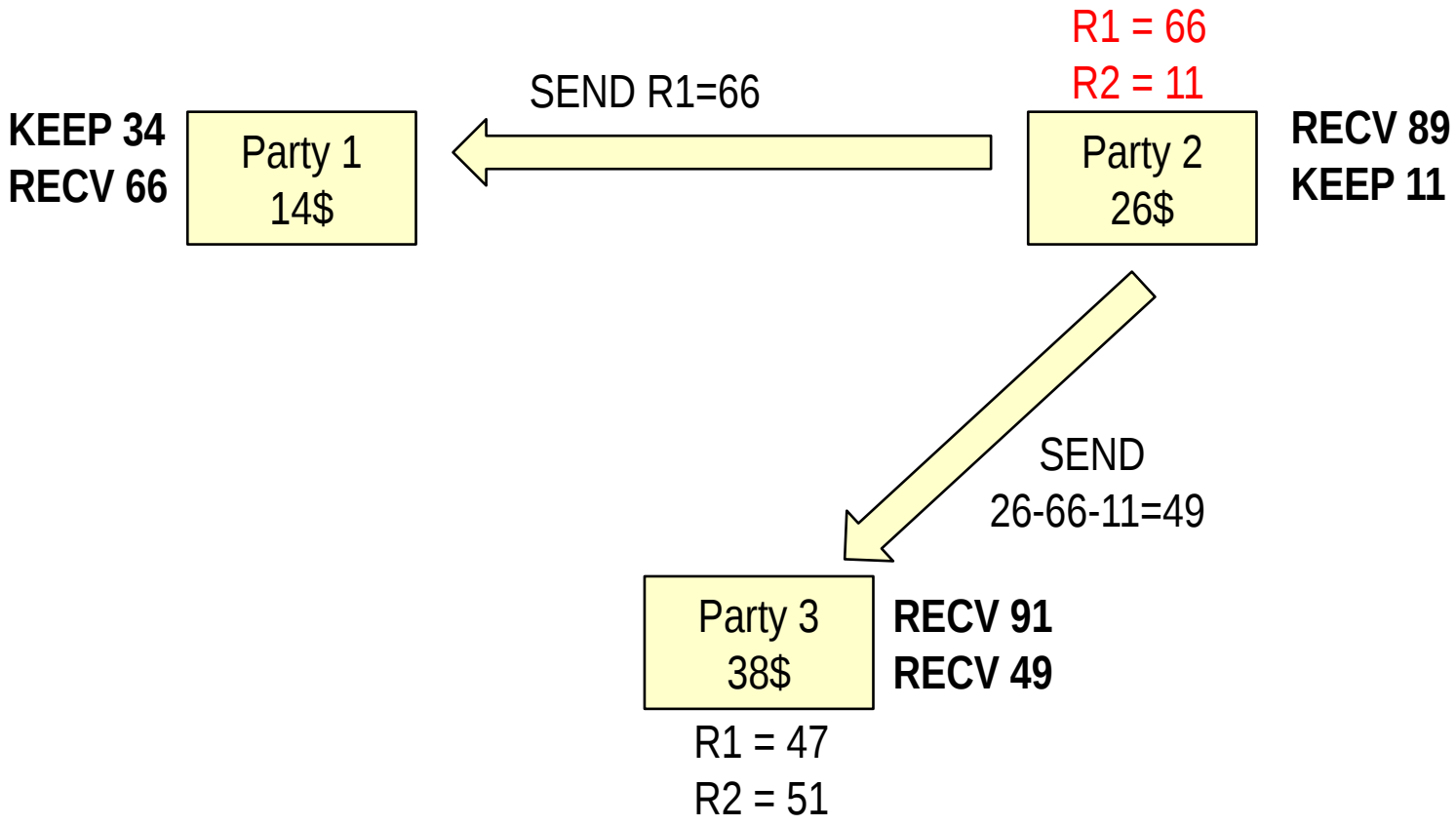
Assume modulo 100 arithmetics

Example
Imput peers = privacy peers



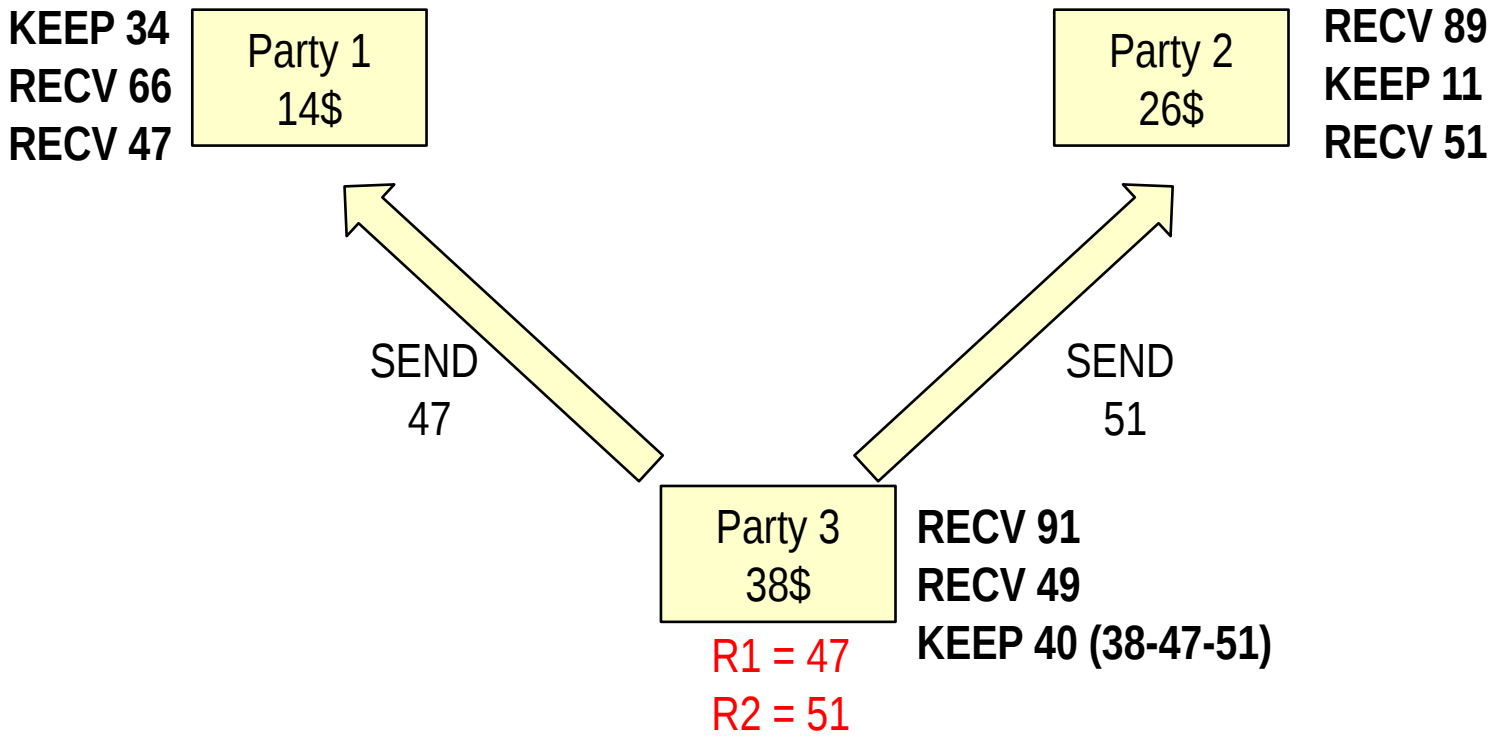
Assume modulo 100 arithmetics

Example
Imput peers = privacy peers



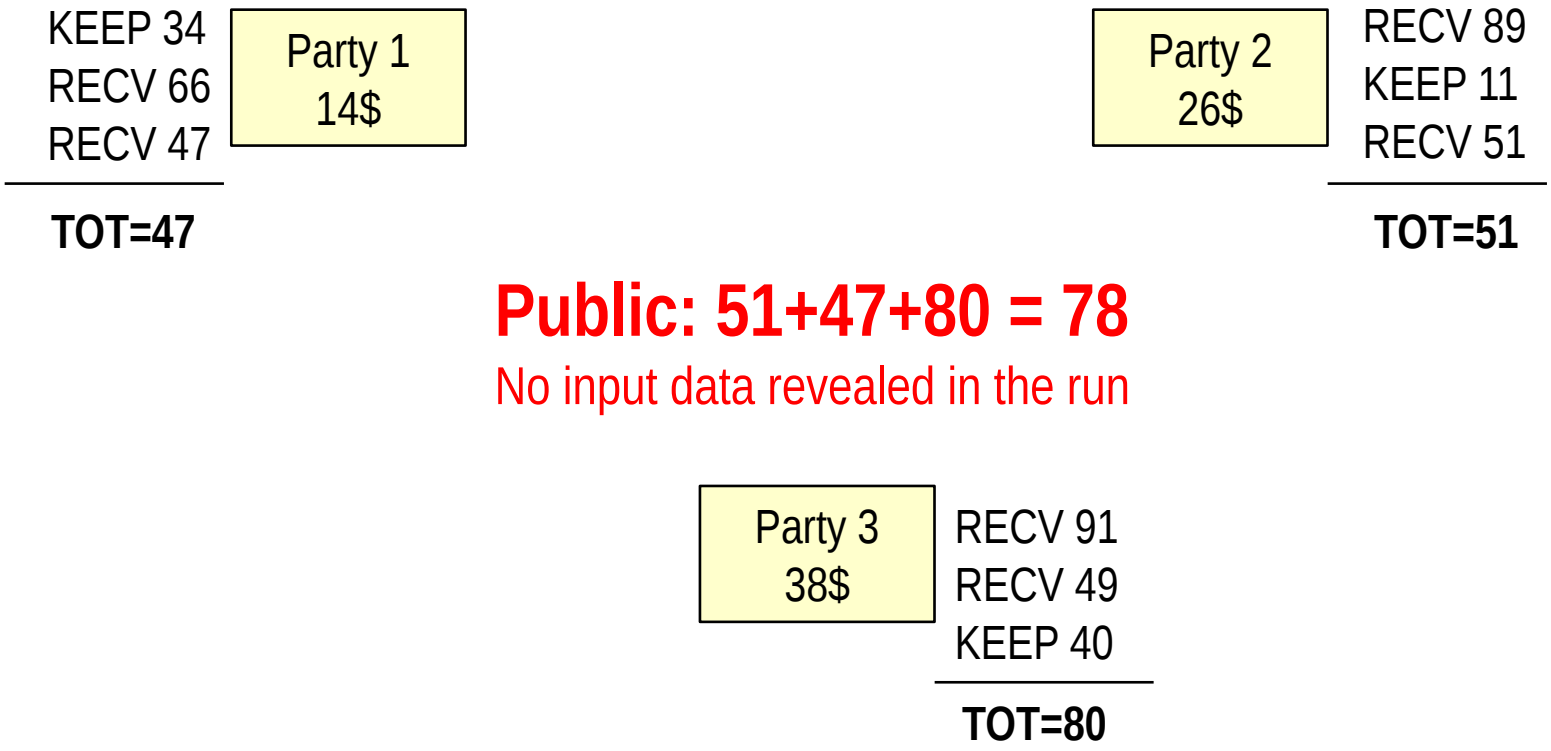
Assume modulo 100 arithmetics

Example
Imput peers = privacy peers



Assume modulo 100 arithmetics

Example
Imput peers = privacy peers



Assume modulo 100 arithmetics